

Doc. No.:	P0941R2
Date:	2018-06-08
Reply to:	Ville Voutilainen Jonathan Wakely
Audience:	CWG, LWG

Integrating feature-test macros into the C++ WD (rev. 2)

[-1] Preface

This revision of this paper proposes incorporating feature-testing macros into the working draft of the C++ standard.

[0] Contents

1. [Introduction](#)
2. [Wording](#)
 1. [Testing for the presence of an attribute: `\_\_has\_cpp\_attribute`](#)
 2. [Feature-testing macros](#)
3. [Feedback question on RTTI and exceptions](#)
4. [Revision history](#)

[1] Introduction

At the November 2017 (Albuquerque) meeting of WG21, there was a five-way poll of all of the C++ experts in attendance concerning their support for "Feature macros should be standardized in the IS, and we want to invite a proposal to be reviewed by EWG to discuss the rationale and granularity." The results of the poll:

Strongly favor	Favor	Neutral	Oppose	Strongly oppose
27	21	9	4	1

The approach taken here is as follows:

- For the macros incorporated, use the latest value from SD-6, and do not bump the numbers. This is done in order to avoid pointless breakage of existing code that uses the SD-6 macros.
- Library macros are defined both in the corresponding header for the library facility and in `<version>`.

[2] Wording

[2.1] Testing for the presence of an attribute: `__has_cpp_attribute`

In Clause [cpp.cond] add the following content:

```
has-include-expression:
    __has_include ( < h-char-sequence > )
```

```
__has_include ( " q-char-sequence " )
__has_include ( string-literal )
__has_include ( < h-pp-tokens > )
```

has-attribute-expression:

```
__has_cpp_attribute ( _pp-tokens ).
```

-1- The expression that controls conditional inclusion shall be an integral constant expression except that identifiers (including those lexically identical to keywords) are interpreted as described below and it may contain zero or more *defined-macro-expressions* and/or *has-include-expressions* and/or *has-attribute-expressions* as unary operator expressions.

-2- A *defined-macro-expression* evaluates to 1 if the identifier is currently defined as a macro name (that is, if it is predefined or if it has been the subject of a #define preprocessing directive without an intervening #undef directive with the same subject identifier), 0 if it is not.

-3- The third and fourth forms of *has-include-expression* are considered only if neither of the first or second forms matches, in which case the preprocessing tokens are processed just as in normal text.

-4- The header or source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a #include directive, except that no further macro expansion is performed. If such a directive would not satisfy the syntactic requirements of a #include directive, the program is ill-formed. The *has-include-expression* evaluates to 1 if the search for the source file succeeds, and to 0 if the search fails.

[-?- Each *has-attribute-expression* is replaced by a non-zero *pp-number* matching the form of an *integer-literal* if the implementation supports an attribute with the name specified by interpreting the *pp-tokens* as an *attribute-token*, and by 0 otherwise. The program is ill-formed if the *pp-tokens* do not match the form of an *attribute-token*.

[-?- For an attribute specified in this document, the value of the *has-attribute-expression* is given by Table ?. For other attributes recognized by the implementation, the value is implementation-defined. [Note: It is expected that the availability of an attribute can be detected by any non-zero result. — end note].

The #ifdef and #ifndef directives, and the defined conditional inclusion operator, shall treat __has_include and __has_cpp_attribute as if ~~it~~ they were the names of ~~a~~ defined macros. The identifiers __has_include and __has_cpp_attribute shall not appear in any context not mentioned in this subclause.

Table ? - __has_cpp_attribute values

<u>Attribute</u>	<u>Value</u>
<u>carries_dependency</u>	<u>200809L</u>
<u>deprecated</u>	<u>201309L</u>
<u>fallthrough</u>	<u>201603L</u>
<u>likely</u>	<u>201803L</u>
<u>maybe_unused</u>	<u>201603L</u>
<u>no_unique_address</u>	<u>201803L</u>
<u>nodiscard</u>	<u>201603L</u>
<u>noreturn</u>	<u>200809L</u>

unlikely	201803L
----------	---------

Adjust the example at the end of [cpp.cond] and add a new example:

[Example: This demonstrates a way to include a library optional facility only if it is available:

```
#if __has_include(<optional>)
# include <optional>
# if cpp_lib_optional >= 201603
#_ define have_optional 1
# endif
#elif __has_include(<experimental/optional>)
# include <experimental/optional>
# if cpp_lib_experimental_optional >= 201411
#_ define have_optional 1
#_ define experimental_optional 1
# endif
# endif
# else #ifndef have_optional
# define have_optional 0
# endif
```

—end example]

[Example: This demonstrates a way to use the attribute `[[acme::deprecated]]` only if it is available.

```
#if __has_cpp_attribute(acme::deprecated)
# define ATTR_DEPRECATED(msg) [[acme::deprecated(msg)]]
# else
# define ATTR_DEPRECATED(msg) [[deprecated(msg)]]
# endif
ATTR_DEPRECATED("This function is deprecated") void anvil();
```

—end example].

[2.2] Feature-testing macros

Drafting note: these are intended to match the latest values in SD-6. Wording review should check that that is indeed so.

In [cpp.predefined] add the following content and table:

-1-The following macro names shall be defined by the implementation:

— __cplusplus

...

— __DATE__

...

— __FILE__

...

— __LINE__

...

— __STDC_HOSTED__

...

— __STDCPP_DEFAULT_NEW_ALIGNMENT__

...

— __TIME__

...

— The names listed in Table ??.

The macros defined in Table ?? shall be defined to the corresponding integer literal. *[Note: Future versions of this International Standard might replace the values of these macros with greater values. —end note].*

Table ?? - Feature-test macros	
Name	Value
<code>__cpp_aggregate_bases</code>	<code>201603L</code>
<code>__cpp_aggregate_nsdmi</code>	<code>201304L</code>
<code>__cpp_alias_templates</code>	<code>200704L</code>
<code>__cpp_aligned_new</code>	<code>201606L</code>
<code>__cpp_attributes</code>	<code>200809L</code>
<code>__cpp_binary_literals</code>	<code>201304L</code>
<code>__cpp_capture_star_this</code>	<code>201603L</code>
<code>__cpp_constexpr</code>	<code>201603L</code>
<code>__cpp_decltype</code>	<code>200707L</code>
<code>__cpp_decltype_auto</code>	<code>201304L</code>
<code>__cpp_deduction_guides</code>	<code>201703L</code>
<code>__cpp_delegating_constructors</code>	<code>200604L</code>
<code>__cpp_enumerator_attributes</code>	<code>201411L</code>
<code>__cpp_fold_expressions</code>	<code>201603L</code>
<code>__cpp_generic_lambdas</code>	<code>201304L</code>
<code>__cpp_guaranteed_copy_elision</code>	<code>201606L</code>
<code>__cpp_hex_float</code>	<code>201603L</code>
<code>__cpp_if_constexpr</code>	<code>201606L</code>
<code>__cpp_inheriting_constructors</code>	<code>201511L</code>
<code>__cpp_init_captures</code>	<code>201304L</code>
<code>__cpp_initializer_lists</code>	<code>200806L</code>
<code>__cpp_inline_variables</code>	<code>201606L</code>
<code>__cpp_lambdas</code>	<code>200907L</code>
<code>__cpp_namespace_attributes</code>	<code>201411L</code>
<code>__cpp_noexcept_function_type</code>	<code>201510L</code>
<code>__cpp_nontype_template_args</code>	<code>201411L</code>
<code>__cpp_nontype_template_parameter_auto</code>	<code>201606L</code>
<code>__cpp_nsdmi</code>	<code>200809L</code>
<code>__cpp_range_based_for</code>	<code>201603L</code>
<code>__cpp_raw_strings</code>	<code>200710L</code>
<code>__cpp_ref_qualifiers</code>	<code>200710L</code>

<u>Name</u>	<u>Value</u>
<u>__cpp_return_type_deduction</u>	<u>201304L</u>
<u>__cpp_rvalue_references</u>	<u>200610L</u>
<u>__cpp_sized_deallocation</u>	<u>201309L</u>
<u>__cpp_static_assert</u>	<u>201411L</u>
<u>__cpp_structured_bindings</u>	<u>201606L</u>
<u>__cpp_template_template_args</u>	<u>201611L</u>
<u>__cpp_threadsafe_static_init</u>	<u>200806L</u>
<u>__cpp_unicode_characters</u>	<u>200704L</u>
<u>__cpp_unicode_literals</u>	<u>200710L</u>
<u>__cpp_user_defined_literals</u>	<u>200809L</u>
<u>__cpp_variable_templates</u>	<u>201304L</u>
<u>__cpp_variadic_templates</u>	<u>200704L</u>
<u>__cpp_variadic_using</u>	<u>201611L</u>

Add the following content to [support.limits.general]:

The macros in Table ??? are defined after inclusion of the header <version> or one of the corresponding headers specified in the table. [Note: Future versions of this International Standard might replace the values of these macros with greater values. —end note].

Table ??? - Standard library feature-test macros

<u>Macro name</u>	<u>Value</u>	<u>Headers</u>
<u>__cpp_lib_addressof_constexpr</u>	<u>201603L</u>	<u><memory></u>
<u>__cpp_lib_allocator_traits_is_always_equal</u>	<u>201411L</u>	<u><memory></u> <u><scoped_allocator></u> <u><string></u> <u><deque></u> <u><forward_list></u> <u><list></u> <u><vector></u> <u><map></u> <u><set></u> <u><unordered_map></u> <u><unordered_set></u>
<u>__cpp_lib_any</u>	<u>201606L</u>	<u><any></u>
<u>__cpp_lib_apply</u>	<u>201603L</u>	<u><tuple></u>
<u>__cpp_lib_array_constexpr</u>	<u>201603L</u>	<u><iterator></u> <u><array></u>
<u>__cpp_lib_as_const</u>	<u>201510L</u>	<u><utility></u>
<u>__cpp_lib_atomic_is_always_lock_free</u>	<u>201603L</u>	<u><atomic></u>
<u>__cpp_lib_bool_constant</u>	<u>201505L</u>	<u><type_traits></u>
<u>__cpp_lib_boyer_moore_searcher</u>	<u>201603L</u>	<u><functional></u>
<u>__cpp_lib_byte</u>	<u>201603L</u>	<u><cstdint></u>
<u>__cpp_lib_chrono</u>	<u>201611L</u>	<u><chrono></u>
<u>__cpp_lib_clamp</u>	<u>201603L</u>	<u><algorithm></u>
<u>__cpp_lib_complex_udls</u>	<u>201309L</u>	<u><complex></u>
<u>__cpp_lib_enable_shared_from_this</u>	<u>201603L</u>	<u><memory></u>
<u>__cpp_lib_exchange_function</u>	<u>201304L</u>	<u><utility></u>
<u>__cpp_lib_execution</u>	<u>201603L</u>	<u><execution></u>
<u>__cpp_lib_filesystem</u>	<u>201703L</u>	<u><filesystem></u>
<u>__cpp_lib_gcd_lcm</u>	<u>201606L</u>	<u><numeric></u>
<u>__cpp_lib_generic_associative_lookup</u>	<u>201304L</u>	<u><map></u> <u><set></u>
<u>__cpp_lib_hardware_interference_size</u>	<u>201703L</u>	<u><new></u>
<u>__cpp_lib_has_unique_object_representations</u>	<u>201606L</u>	<u><type_traits></u>
<u>__cpp_lib_hypot</u>	<u>201603L</u>	<u><cmath></u>
<u>__cpp_lib_incomplete_container_elements</u>	<u>201505L</u>	<u><forwardlist></u> <u><list></u> <u><vector></u>
<u>__cpp_lib_integer_sequence</u>	<u>201304L</u>	<u><utility></u>
<u>__cpp_lib_integral_constant_callable</u>	<u>201304L</u>	<u><type_traits></u>
<u>__cpp_lib_invoke</u>	<u>201411L</u>	<u><functional></u>
<u>__cpp_lib_is_aggregate</u>	<u>201703L</u>	<u><type_traits></u>
<u>__cpp_lib_is_final</u>	<u>201402L</u>	<u><type_traits></u>
<u>__cpp_lib_is_invocable</u>	<u>201703L</u>	<u><type_traits></u>
<u>__cpp_lib_is_null_pointer</u>	<u>201309L</u>	<u><type_traits></u>
<u>__cpp_lib_is_swappable</u>	<u>201603L</u>	<u><type_traits></u>
<u>__cpp_lib_laundry</u>	<u>201606L</u>	<u><new></u>

<u>__cpp_lib_logical_traits</u>	<u>201510L</u>	<u><type_traits></u>
<u>__cpp_lib_make_from_tuple</u>	<u>201606L</u>	<u><tuple></u>
<u>__cpp_lib_make_reverse_iterator</u>	<u>201402L</u>	<u><iterator></u>
<u>__cpp_lib_make_unique</u>	<u>201304L</u>	<u><memory></u>
<u>__cpp_lib_map_try_emplace</u>	<u>201411L</u>	<u><map></u>
<u>__cpp_lib_math_special_functions</u>	<u>201603L</u>	<u><cmath></u>
<u>__cpp_lib_memory_resource</u>	<u>201603L</u>	<u><memory_resource></u>
<u>__cpp_lib_node_extract</u>	<u>201606L</u>	<u><map> <set></u> <u><unordered map></u> <u><unordered set></u>
<u>__cpp_lib_nonmember_container_access</u>	<u>201411L</u>	<u><iterator> <array></u> <u><deque> <forward_list></u> <u><list> <map> <regex></u> <u><set> <string></u> <u><unordered map></u> <u><unordered set> <vector></u>
<u>__cpp_lib_not_fn</u>	<u>201603L</u>	<u><functional></u>
<u>__cpp_lib_null_iterators</u>	<u>201304L</u>	<u><iterator></u>
<u>__cpp_lib_optional</u>	<u>201606L</u>	<u><optional></u>
<u>__cpp_lib_parallel_algorithm</u>	<u>201603L</u>	<u><algorithm> <numeric></u>
<u>__cpp_lib_quoted_string_io</u>	<u>201304L</u>	<u><iomanip></u>
<u>__cpp_lib_raw_memory_algorithms</u>	<u>201606L</u>	<u><memory></u>
<u>__cpp_lib_result_of_sfinae</u>	<u>201210L</u>	<u><functional></u> <u><type_traits></u>
<u>__cpp_lib_robust_nonmodifying_seq_ops</u>	<u>201304L</u>	<u><algorithm></u>
<u>__cpp_lib_sample</u>	<u>201603L</u>	<u><algorithm></u>
<u>__cpp_lib_scoped_lock</u>	<u>201703L</u>	<u><mutex></u>
<u>__cpp_lib_shared_mutex</u>	<u>201505L</u>	<u><shared_mutex></u>
<u>__cpp_lib_shared_ptr_arrays</u>	<u>201611L</u>	<u><memory></u>
<u>__cpp_lib_shared_ptr_weak_type</u>	<u>201606L</u>	<u><memory></u>
<u>__cpp_lib_shared_timed_mutex</u>	<u>201402L</u>	<u><shared_mutex></u>
<u>__cpp_lib_string_udls</u>	<u>201304L</u>	<u><string></u>
<u>__cpp_lib_string_view</u>	<u>201606L</u>	<u><string> <string_view></u>
<u>__cpp_lib_to_chars</u>	<u>201611L</u>	<u><utility></u>
<u>__cpp_lib_transformation_trait_aliases</u>	<u>201304L</u>	<u><type_traits></u>
<u>__cpp_lib_transparent_operators</u>	<u>201510L</u>	<u><memory> <functional></u>
<u>__cpp_lib_tuple_element_t</u>	<u>201402L</u>	<u><tuple></u>
<u>__cpp_lib_tuples_by_type</u>	<u>201304L</u>	<u><utility> <tuple></u>
<u>__cpp_lib_type_trait_variable_templates</u>	<u>201510L</u>	<u><type_traits></u>
<u>__cpp_lib_uncaught_exceptions</u>	<u>201411L</u>	<u><exception></u>
<u>__cpp_lib_unordered_map_try_emplace</u>	<u>201411L</u>	<u><unordered_map></u>

<u>__cpp_lib_variant</u>	<u>201606L</u>	<u><variant></u>
<u>__cpp_lib_void_t</u>	<u>201411L</u>	<u><type_traits></u>

[3] Revision history

Date	Document	Description
2018-02-09	P0941R0	Initial draft
2018-05-04	P0941R1	Draft wording, with non-standard content removed and rtti/exception macros made a separate point looking for feedback.
2018-06-08	P0941R2	Removed __cpp_rtti and __cpp_exceptions. Moved and rewrote proposed wording. Used separate tables for attributes, core macros, and library macros. Sorted tables by attribute/macro name. Added new attributes approved in Jacksonville meeting. Removed duplicate entries for macros. Fixed header for __cpp_lib_tuple_element_t. Added <tuple> header for __cpp_lib_tuples_by_type. Updated value for __cpp_deduction_guides.